

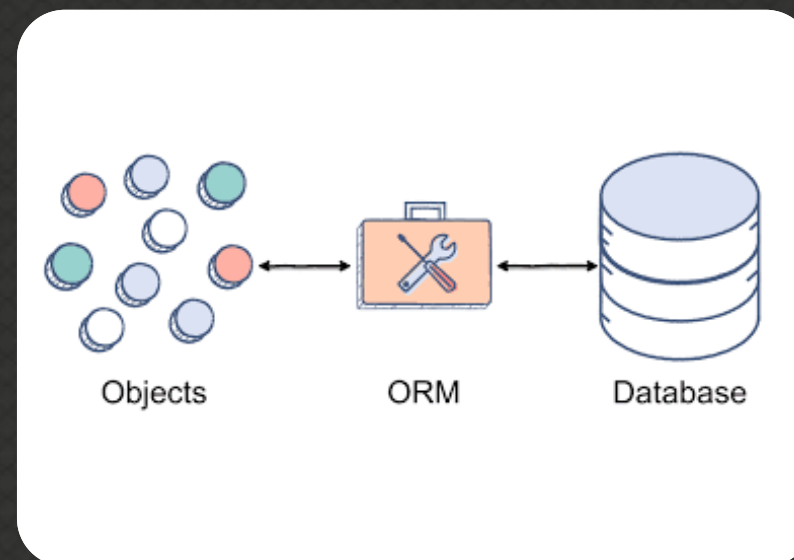
Aula 06:

Conexão ao banco de dados MySQL com Sequelize

O que é uma ORM?

Uma **ORM**, ou **Object-Relational Mapping**, é uma técnica de programação que visa facilitar a interação entre um **SGBDR** (*sistema de gerenciamento de banco de dados relacional, como MySQL, PostgreSQL, SQLite, etc.*) e um **programa orientado a objetos**.

A ideia principal é **mapear objetos** do código-fonte para **tabelas** no banco de dados e vice-versa, eliminando a necessidade de escrever **consultas SQL** manualmente e permitindo que os desenvolvedores usem **objetos** e **métodos** para manipular dados.



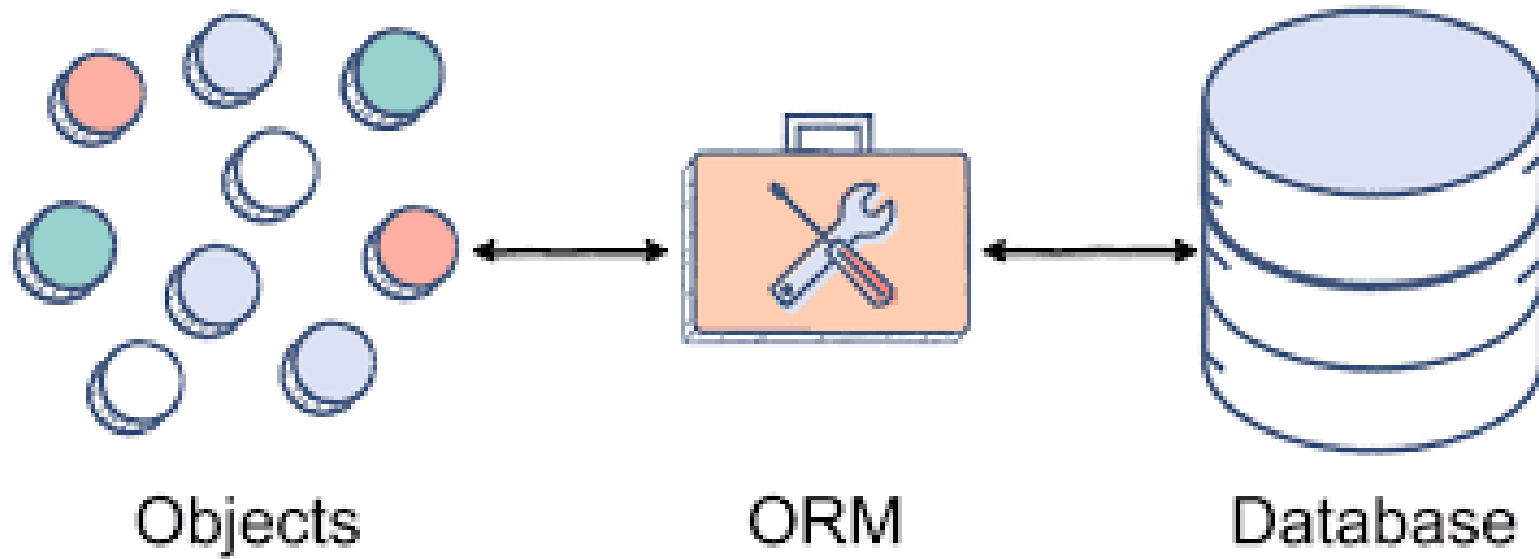
{ }

JS

O

Um
Ma
que
SGI
ban
Pos
orie

A id
cóo
dad
ne
ma
des
mé



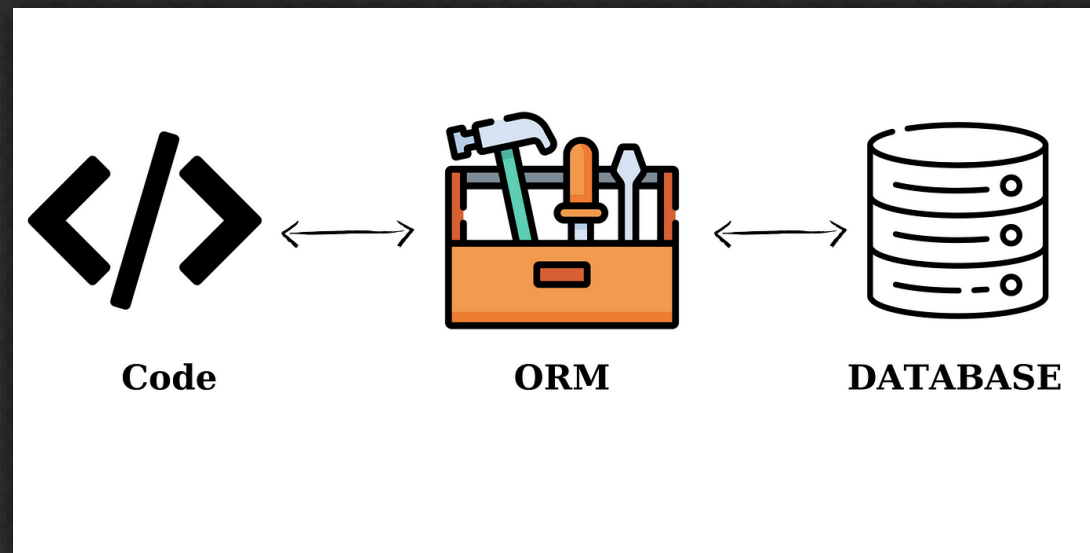
{ }

JS

O que é uma ORM?

As ORMs desempenham um papel crucial em ambientes onde o código é predominantemente **orientado a objetos**, como em muitas linguagens de programação modernas.

Elas ajudam a abstrair a complexidade das operações de banco de dados, proporcionando uma interface mais intuitiva e facilitando a manutenção do código.





Query SQL

```
SELECT * FROM clientes
```

Método da ORM (Sequelize)

```
Cliente.findAll()
```

JS

ORM Sequelize

O **Sequelize** é uma **ORM** para **Node.js** que suporta vários bancos de dados relacionais, incluindo MySQL, PostgreSQL, SQLite, etc.

O Sequelize facilita a interação entre **SGBDR** e código **JavaScript**, permitindo que os desenvolvedores usem objetos e métodos para manipular dados em vez de escrever consultas SQL manualmente.

Para instalar o Sequelize no seu projeto Node, basta rodar o comando:

```
npm install sequelize
```



JS

MySQL2

O `mysql2` é um driver MySQL para `Node.js`. Em termos simples, é uma biblioteca que permite que você se conecte a um banco de dados MySQL a partir do ambiente Node.js e execute consultas SQL de forma programática.

Para instalar o MySQL2 no seu projeto Node, basta rodar o comando:

```
npm install mysql2
```

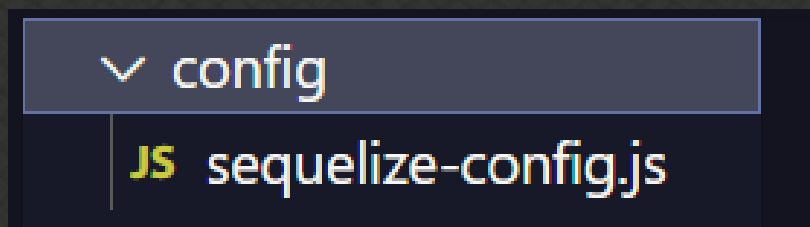


JS

Instalando e configurando o Sequelize

Para instalar o Sequelize no seu projeto utilize o comando: **npm install sequelize**

Após isso, com o Sequelize instalado no seu projeto, criaremos uma nova pasta com o nome **“config”** dentro da pasta do seu projeto. Dentro desta pasta criaremos o arquivo de configuração do Sequelize, com o nome **“sequelize-config.js”**



Configurando o Sequelize

No arquivo **“sequelize-config.js”**, faremos a importação do Sequelize e em seguida iremos inserir as configurações de conexão com o banco de dados, como endereço do banco, usuário, senha, nome do banco, etc.

```
// Importando o Sequelize
import Sequelize from "sequelize"

// Criando os dados de conexão com o banco de dados
const connection = new Sequelize({
  dialect: 'mysql',
  host: 'localhost',
  username: 'root',
  password: 'admin',
  // Comente essa linha na primeira execução da aplicação
  // database: 'loja',
  timezone: "-03:00"
})

export default connection
```



JS

Criando a conexão com o banco de dados



Com o arquivo de configuração criado, no arquivo **"index.js"** iremos começar a criar as linhas responsáveis pela conexão da aplicação com o banco. O primeiro passo é importar o arquivo de configuração do Sequelize:

```
// Importando o Sequelize
import connection from './config/sequelize-config.js'
```

Após isso, criaremos a conexão com o banco de dados:

```
// Realizando a conexão com o banco de dados
connection.authenticate().then(() => {
  console.log("Conexão com o banco de dados feita com sucesso!")
}).catch((error) => {
  console.log(error)
})
```

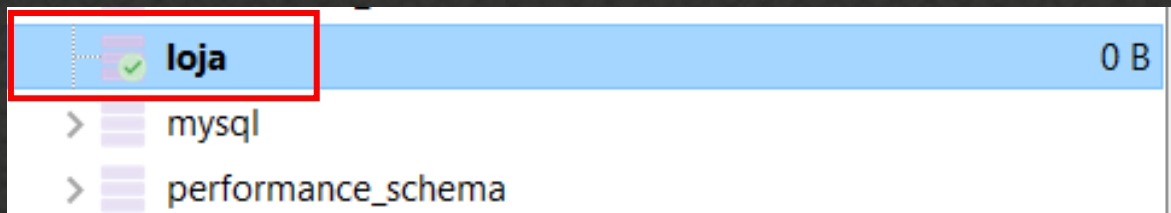
JS

Criando o banco de dados

Logo abaixo das linhas que criam a conexão com o banco, iremos inserir as linhas responsáveis por criar o banco de dados caso ele ainda não exista.

```
// Criando o banco de dados se ele não existir
connection.query(`CREATE DATABASE IF NOT EXISTS
loja;`).then(() => {
  console.log("O banco de dados está criado.")
}).catch((error) => {
  console.log(error)
})
```

Feito isso, ao rodarmos nossa aplicação, já podemos conferir se o banco de dados foi devidamente criado.



JS

Configurando o Sequelize

Agora que o banco de dados já foi criado, devemos voltar ao arquivo **“sequelize-config.js”** e retirar o comentário da linha **“database”** que indica qual banco será usado pela aplicação. Iniciamos com essa linha comentado anteriormente pois o banco ainda não existia e retornaria um erro.

```
// Importando o Sequelize
import Sequelize from "sequelize"

// Criando os dados de conexão com o banco de dados
const connection = new Sequelize({
  dialect: 'mysql',
  host: 'localhost',
  username: 'root',
  password: 'admin',
  // Comente essa linha na primeira execução da aplicação
  database: 'loja',
  timezone: "-03:00"
})

export default connection
```



JS

Editando o Model

Agora em nosso arquivo de model **Cliente.js**, incluiremos o seguinte código:

```
import Sequelize from "sequelize"
import connection from "../config/sequelize-
config.js"
```

1 - Importação do Sequelize e seu arquivo de configuração.

```
const Cliente = connection.define('clientes',
{
  nome:{
    type: Sequelize.STRING,
    allowNull: false
  },
  cpf:{
    type: Sequelize.STRING,
    allowNull: false
  },
  endereco:{
    type: Sequelize.STRING,
    allowNull: false
  }
})
```

2 – Criação de uma nova entidade no banco com o nome **“clientes”** que será armazenado na variável **Cliente**, contendo os campos **nome**, **cpf** e **endereço**, todos do tipo **String**.

3 – Linha que cria a entidade no banco, com a opção “force” definida com o valor “false” ou seja, caso a tabela já exista não será criada novamente.

```
Cliente.sync({force: false})
```

```
export default Cliente
```

4 - Exportação do módulo.



JS

Editando o Model

Aqui estão alguns dos tipos de dados comuns suportados pelo Sequelize:

STRING: Uma string de caracteres variáveis.

CHAR: Uma string de caracteres fixos.

TEXT: Uma string de texto longo.

INTEGER: Um número inteiro.

FLOAT: Um número decimal de precisão simples.

DOUBLE: Um número decimal de precisão dupla.

BOOLEAN: Um valor booleano (true/false).

DATE: Uma data.

TIME: Um valor de tempo.

ARRAY: Uma matriz de valores de qualquer tipo de dados suportado pelo Sequelize.

JSON: Um tipo de dados JSON.



JS

Inserindo registros

Com o nosso arquivo de model pronto, basta o importarmos no arquivo

ClienteControllers.js:

```
// Importando os Models
import Cliente from "../models/Cliente.js"
```

Agora ao rodar aplicação a tabela “clientes” já deve estar criada em nosso banco de dados:

loja	16,0 KiB
clientes	16,0 KiB
mysql	
performance_schema	



JS

Criando uma tabela no banco

Note que todos os campos que definimos no arquivo de model, foram criados na tabela. Além deles, foram criados também os campos **“createdAt”** e **“updatedAt”** que já são campos criados pelo próprio Sequelize:

Colunas: + Adicionar × Remover ▲ Mover para cima ▼ Mover para baixo							
#	Nome	Tipo de dados	Tamanho/Itens	Unsigned	Permitir ...	Zerofill	Padrão
1	id	INT	10	☐	☐	☐	AUTO_INCREME...
2	nome	VARCHAR	255	☐	☐	☐	Nenhum padrão
3	cpf	VARCHAR	255	☐	☐	☐	Nenhum padrão
4	endereco	VARCHAR	255	☐	☐	☐	Nenhum padrão
5	createdAt	DATETIME		☐	☐	☐	Nenhum padrão
6	updatedAt	DATETIME		☐	☐	☐	Nenhum padrão

Para testarmos a busca de dados no banco, iremos inserir alguns registros de forma manual, para que quando acessarmos a rota de clientes alguns registros sejam mostrados.

id	nome	cpf	endereco	createdAt	updatedAt
1	José Ribeiro	999.999.999-00	Rua Perdizes, 230- Jardim...	2024-01-18 16:19:42	2024-01-18 16:19:45
2	Maria Pereira	555.555.555-00	Rua das Árvores, 148 - ...	2024-01-18 16:26:23	2024-01-18 16:26:25